

Qwen3-Coder-Next Tier 3

Part 10 Detailed Implementation

Specification

Code Knowledge Graph

Engineering specification for implementing a repository-wide dependency and impact graph

Part 10

Foundation for repository reasoning, impact analysis, and graph-driven agent behavior

1. Purpose

The Code Knowledge Graph exists to give the agent a live, queryable model of how a repository is wired together. It captures symbols, dependencies, imports, call paths, ownership signals, and change impact so agents can reason about relationships instead of raw files.

This part solves the classic failure mode of file-centric automation: an agent sees one file, edits it, and silently breaks three other modules that were never loaded into context. The graph removes that blindness by making structure explicit and searchable.

This part depends on the filesystem and local tooling foundation, repository intelligence inputs, and the earlier planning, research, memory, testing, and review layers. Later parts consume it for context compression, evaluation, recovery, and multi-agent coordination.

Key rule: If the graph cannot answer "what depends on this?" with a stable result, it is not complete enough to support autonomous code work.

2. Scope

Included

- Repository scanning and file classification.
- Language parser adapters and symbol extraction.
- Node and edge normalization across languages.
- Persistent graph storage and snapshot management.
- Incremental updates for changed files and renamed symbols.
- Query APIs for dependency lookup, call-path tracing, and blast-radius analysis.
- Export hooks for inspection, debugging, and downstream tooling.

Excluded

- Vector search and semantic retrieval.
- Context compression summaries.
- LLM prompt construction logic.
- Code editing, testing, or deployment behavior.
- Benchmark reporting and observability dashboards beyond basic graph metrics.

3. System Overview

User / Agent Request

↓

Planner or Research Layer

↓

Repository Intelligence Intake

↓

Scanner → Parser Adapters → Relation Extraction

↓

Graph Builder / Normalizer

↓

Persistent Graph Store + Snapshot Index

↓

Query API / Impact Analysis / Subgraph Export

↓

Planning, Review, Recovery, and Context Compression

The complete flow starts when the repository changes or an agent asks a structural question. The scanner discovers affected files, parser adapters extract symbols and relationships, the normalizer assigns canonical IDs, and the graph store persists the result. Consumers then query the graph for dependency chains, call paths, ownership hints, and affected areas before acting.

4. Internal Architecture

Core components and responsibilities

Component	Responsibility	Primary dependency
Repository Scanner	Detects files, directories, change sets, and ignored paths.	Filesystem + ignore rules
Parser Adapter	Turns source text into symbols, references, imports, and calls.	Language-specific parser
Relation Extractor	Normalizes AST facts into graph nodes and edges.	Symbol table + canonical naming
Graph Builder	Assembles a stable graph snapshot from extracted relations.	Node and edge schema
Graph Store	Persists graphs, deltas, and lookup indexes.	Database or local store
Query Engine	Serves dependency, path, and impact queries.	Snapshot index + traversal rules
Invalidation Manager	Marks stale regions after file edits or symbol renames.	Change events
Export Layer	Produces JSON/CSV/text subgraphs for debugging.	Query engine output

Required abstractions

- SourceUnit: one file plus metadata such as path, language, hash, and revision.

- **GraphNode**: canonical entity such as file, module, class, function, variable, or external dependency.
- **GraphEdge**: typed relationship such as imports, calls, defines, references, contains, or depends-on.
- **GraphSnapshot**: immutable indexed version of the graph at a point in time.
- **GraphQuery**: structured request describing traversal depth, edge types, filters, and scope.

High-level interaction model

SourceUnit -> Parser Adapter -> Symbol Facts

Symbol Facts -> Relation Extractor -> GraphNode / GraphEdge records

GraphNode / GraphEdge records -> Graph Builder -> Snapshot

Snapshot -> Query Engine -> Subgraph / Path / Impact Report

Design constraint: Every graph entity must have a canonical ID derived from repository scope, language, and symbol path. Unstable IDs make incremental updates and comparisons impossible.

5. Project Structure

Folder	Purpose
graph/	Graph schema, nodes, edges, traversal utilities, and snapshot model.
scanner/	Filesystem discovery, ignore filtering, file hashing, and change detection.
parsers/	Language adapters and AST-to-facts extraction logic.
extractors/	Normalization of relations, symbol resolution, and deduplication.
storage/	Persistent store adapters, indexes, and snapshot persistence.
query/	Traversal engine, impact analysis, and query composition.
invalidation/	Dirty-path tracking, rename handling, and stale-region cleanup.
export/	JSON/CSV/text export and debugging helpers.
api/	Public service interface for agents and tooling.
config/	Supported languages, ignore patterns, limits, and storage settings.
tests/	Fixtures, golden graphs, incremental update tests,

	and query verification.
--	-------------------------

The structure keeps extraction logic separate from persistence and query concerns. That separation matters because the graph will evolve; the schema and traversal rules should not be entangled with parser details.

6. Data Contracts

Input structures

Name	Fields
FileScanEvent	path, repo_id, revision_id, hash, language, change_type
ParseRequest	source_unit, parser_profile, scope, revision_id
GraphQueryRequest	repo_id, revision_id, node_filters, edge_filters, depth, direction, limit
InvalidationEvent	repo_id, touched_paths, renamed_symbols, revision_id

Output structures

Name	Fields
GraphNode	node_id, kind, name, qualified_name, file_path, language, revision_id
GraphEdge	edge_id, kind, from_node, to_node, confidence, provenance
SubgraphResult	nodes, edges, root_nodes, traversal_metadata
ImpactReport	changed_paths, affected_nodes, downstream_depth, blast_radius_summary

Artifact and configuration formats

- Graph snapshots must be serializable to a stable JSON format for reproducibility.
- Edge and node exports should preserve provenance so consumers can inspect where each fact came from.
- Configuration must define language support, ignore patterns, store backend, traversal limits, and incremental update policy.

Contract rule: A query result must be reproducible from the same snapshot. If the same request returns different structure without the snapshot changing, the system is leaking non-

determinism.

7. State Management

The graph service owns persistent graph state. It maintains the latest committed snapshot, delta history for incremental updates, and a dirty queue for files that need reprocessing.

Working state includes the current revision, pending scan results, parser caches, and traversal scratch space. Persistent state includes node and edge indexes, snapshot metadata, and revision-to-snapshot mappings.

Future agents consume state indirectly through query APIs and snapshot references. They should not mutate graph internals directly; all updates must flow through the scan-extract-build pipeline.

- State changes only when a scan event or invalidation event is committed.
- Snapshot IDs are immutable once published.
- Dirty paths are cleared only after their derived nodes and edges are reconciled.
- Cached parser output may be discarded at any time and rebuilt from source.

8. Component Specifications

Component	Purpose	Failure conditions	Future integration points
Scanner	Find files, detect changes, and classify language scope.	Missed files, stale ignore rules, hash mismatches.	Repository intelligence and incremental indexing.
Parser Adapter	Extract symbols and relations from source text.	Unsupported syntax, parse failures, language drift.	Knowledge graph queries and symbol-level retrieval.
Relation Extractor	Convert parser facts into graph entities.	Duplicate symbols, unresolved references, inconsistent naming.	Impact analysis and context compression.
Graph Store	Persist stable snapshots and indexes.	Partial writes, stale snapshots, lost provenance.	Evaluation, recovery, and downstream consumers.
Query Engine	Answer dependency and path questions.	Unbounded traversal, ambiguous filters, empty results from stale state.	Planner, reviewer, evaluator, and recovery agents.
Invalidation Manager	Keep graph consistent after edits.	Rename misses, orphaned nodes, over-	Incremental maintenance and

		invalidation.	rebuild control.
--	--	---------------	------------------

Pseudo-code sketch

```

for changed_file in scan_changes(repo):
    source = load_source(changed_file)
    facts = parse(source)
    relations = normalize(facts)
    graph.apply(relations, revision=current_revision)
publish(snapshot(graph))

```

9. Implementation Sequence

Define the node, edge, snapshot, and query schemas first. This comes first because every later subsystem depends on stable identifiers and typed relations.

Build the scanner and file classification layer. Without reliable input discovery, no downstream graph can be trusted.

Implement parser adapters for the supported language set. This is the first place where syntax is turned into reusable facts.

Add relation normalization and symbol resolution. This removes language-specific noise and creates a canonical graph vocabulary.

Implement the persistent store and snapshot publication flow. The system becomes useful only when graph state survives process restarts.

Add traversal and query APIs for dependency, path, and impact analysis. This is what downstream agents actually consume.

Implement invalidation and incremental rebuild behavior. The graph must remain correct after edits without full reindexing every time.

Add export and inspection tools so engineers can debug graph state without reading storage internals.

Wire the graph into the repository intelligence and agent layers.

Build order rule: Do not add incremental rebuilds before stable schema and traversal queries. Otherwise you will spend your week debugging stale garbage with a smile that says "I hate databases."

10. Validation Strategy

- Unit test each parser adapter against small fixture files with known symbols and references.
- Golden graph tests must compare node and edge sets for sample repositories.
- Incremental update tests must verify that editing one file updates only the expected subgraph.

- Consistency checks must fail when an edge points to a missing node or when duplicate canonical IDs appear.
- Traversal tests must verify depth limits, edge filters, and path ordering.
- Regression tests must compare exported snapshots across builds for stability.

Validation type	What it proves
Parser fixtures	Language-specific extraction is correct.
Golden graph comparison	Schema and normalization are stable.
Incremental replay	Changes propagate without full rebuilds.
Query assertions	Consumers get the right subgraph or impact set.
Store recovery test	Snapshots survive restart and reload correctly.

11. Deliverables

- Graph schema documentation and versioned data contract.
- Scanner and parser adapter modules.
- Relation extraction and normalization pipeline.
- Persistent graph storage adapter.
- Query engine with dependency and impact APIs.
- Invalidation manager and incremental update path.
- Export utilities for JSON and human-readable inspection.
- Test fixtures, golden snapshots, and regression suite.
- Operational configuration for supported languages and limits.

12. Acceptance Criteria

Criterion	How it is verified
A sample repository can be indexed into stable nodes and edges.	Golden graph comparison passes.
A query for "what depends on this symbol" returns the expected downstream set.	Query assertion tests.
Editing a file and reindexing only updates the affected region.	Incremental update test with revision diff.
Snapshot reload restores the same graph shape.	Restart and reload test.
No edge exists without a corresponding node.	Consistency validator passes.
Exported subgraphs match query results.	Export verification against query baseline.

- Acceptance criteria must be data-driven, not qualitative.
- A build is not complete if only parsing works but persistence or traversal fails.
- Every criterion must pass against at least one real fixture repository and one synthetic edge-case repository.

13. Common Failure Modes

- Treating the file tree as the graph. The graph must model symbols and relationships, not just folders.
- Using unstable IDs that change when the file moves or when the parser version changes.
- Over-indexing every token and creating a graph explosion with useless noise.
- Ignoring incremental invalidation, which leaves stale edges after renames or deletions.
- Hard-coding language-specific logic in the store layer.
- Failing to preserve provenance, making debugging impossible.
- Letting traversal queries become unbounded and slow.
- Confusing semantic similarity with structural relation.

14. Future Integration

- Part 11 Context Compression will use graph subgraphs to keep prompt context small and relevant.
- Part 12 Multi-Agent Coordination will use dependency information to split work by module boundaries.
- Part 13 Benchmark Harness will measure traversal latency, rebuild cost, and query accuracy.
- Part 14 Deployment and Human Approval will use blast-radius reports before merge or release actions.
- Recovery and evaluation layers will query the graph to understand failure impact and whether a fix touches the right code paths.

15. Completion Checklist

- ☐ Graph schema is versioned and documented.
- ☐ Scanner, parser, extractor, store, and query modules exist.
- ☐ Supported language fixtures are indexed correctly.
- ☐ Incremental updates produce the expected changed subgraph.
- ☐ Queries return reproducible results from a published snapshot.

- ☐ Exports match stored query output.
- ☐ Consistency validator passes on all fixture repositories.
- ☐ Integration points for later parts are documented.